

PlayPulse — Project Plan

PlayPulse empowers youth sports organizations to run better leagues and tournaments — built for the people who keep the game going.

Four pillars: **Schedule** (games, fields & events) · **Manage** (teams, rosters & RSVPs) · **Compete** (brackets & tournaments) · **Insights** (standings & stats).

This document is the pre-build plan: product scope, personas, data model, API design, screen inventory, architecture, and a phased roadmap. No code yet — this is for review and sign-off.

1. Product vision & positioning

PlayPulse is a **multi-tenant SaaS** for youth sports. A single organization (a club, a league, a tournament host) signs up, then runs everything from registration through final standings in one place. The product replaces the spreadsheet + group-chat + paper-bracket workflow that most youth orgs limp along on today.

Who we beat: TeamSnap (great team chat, weak league/bracket ops), LeagueApps / SportsEngine (powerful but heavyweight and expensive), and the default — Excel + GroupMe + a printed bracket taped to a wall.

Our wedge: fast, modern, mobile-first, and genuinely pleasant to use on game day. The brand promise is *momentum* — the play button becomes a pulse.

Differentiators

- **Game-day first.** Live score entry, instant standings recompute, real-time bracket advancement.
- **One link for everything public.** Schedules, standings, and brackets are shareable, no-login public pages.
- **RSVP that actually works.** Per-game availability for players/parents with auto-reminders and a coach-facing "who's in" view.
- **Tournament engine built in.** Single/double elimination, round-robin, and pool→bracket, with auto-seeding.

2. Personas & roles

PlayPulse is role-based **within each organization** (a user can hold different roles in different orgs).

Role	Who	Primary jobs
Org Admin / Owner	League director, club administrator	Create seasons/divisions, open registration, build schedules, assign venues, manage billing & staff
Coach / Team Manager	Volunteer coach, team parent	Manage roster, set lineups, view & request schedule changes, message team, track RSVPs
Parent / Guardian	Parent of a youth player	RSVP for their child, view schedule & standings, receive notifications, pay registration fees
Player	Youth athlete (older age groups)	View schedule, RSVP, see personal & team stats
Scorekeeper / Official	Referee, table volunteer	Enter scores & game events on game day, confirm results
Spectator (public)	Anyone with the link	View public schedules, standings, brackets — no account required

Permissions are enforced by **(role × scope)** — e.g. a Coach can edit only their own team's roster; an Admin can edit anything in their org.

3. Feature set

3.1 Schedule — *games, fields & events*

- Season / division / round structure
- Auto-schedule generator (round-robin, balanced home/away, venue & blackout constraints)
- Manual drag-and-drop schedule editor
- Venue & field management (multiple fields per venue, time-slot capacity)
- Conflict detection (team double-booked, field double-booked, travel time)
- Non-game events (practices, tryouts, picture day, opening ceremony)
- Reschedule / cancel with automatic notifications
- Calendar sync (iCal/Google subscribe feed per team & per user)

3.2 Manage — *teams, rosters & RSVPs*

- Org → season → division → team hierarchy
- Roster management (players, jersey #, position, guardians)
- Registration & waivers (forms, age-eligibility, payment)
- Per-game RSVP / availability with reminders
- Coach "availability board" (who's in / out / maybe per game)
- Team & org messaging / announcements

- Staff & volunteer assignment (coaches, refs, scorekeepers)

3.3 Compete — *brackets & tournaments*

- Tournament creation: single elim, double elim, round-robin, pool play → bracket
- Seeding (manual, by record, by ranking)
- Auto-advancement as scores are entered
- Pool standings → bracket generation
- Printable / shareable bracket and pool sheets
- Multi-day tournament scheduling with field assignment

3.4 Insights — *standings & stats*

- Live standings (W-L-T, points, goal/point differential, tiebreakers)
- Team & player stat tracking (sport-configurable stat schema)
- Leaderboards
- Game results history & box scores
- Exportable reports (CSV/PDF) for admins
- Org dashboard with season health metrics

3.5 Platform / cross-cutting

- Multi-tenant orgs with custom branding (org logo/colors layered on PlayPulse base)
- Auth: email/password + magic link + Google/Apple SSO
- Notifications: email, web push, optional SMS
- Public no-login pages (schedule, standings, bracket) per share link
- Billing (Stripe) for registration fees and org subscription
- PWA / installable mobile web; offline-tolerant score entry
- Admin audit log

3.6 MVP vs later

MVP (v1) — soccer only: Orgs & roles · seasons → competitions → divisions · clubs/teams/rosters (with player photos) · venues · manual + auto schedule · RSVP · live score entry · live standings · public schedule/standings pages · **email (Resend) + web push** notifications · logo + color org branding. Fully responsive across phone, tablet, and desktop (PWA, no native app).

Registration & payment **bones** are modeled now (`Registration` , `Payment` entities; season-level signup → division placement) so the schema and flows are ready — but the live Stripe checkout is deferred to fast-follow.

Fast-follow (v1.x): Stripe registration payments (activate the bones) · tournament/bracket engine · player stats · calendar sync · reporting/exports.

Later (v2): Additional sports (basketball/baseball + richer stat schemas) · SMS · native mobile apps · league discovery/marketplace · referee assigning & payments · advanced analytics · sponsorships/ads on public pages · officials scheduling.

4. Information architecture (screen inventory)

Admin (desktop-primary):

- Org dashboard (season health, upcoming games, alerts)
- Competitions (leagues & tournaments) & divisions
- Clubs, teams & rosters (with player photos)
- Schedule builder (grid + drag-drop)
- Venues & fields
- Tournaments / brackets
- Standings & stats
- Registration & payments
- Members & roles
- Org settings & branding

Coach (mobile + desktop):

- Today / next game
- My team(s) → roster, RSVPs, lineup
- Schedule
- Messages / announcements
- Score entry (if enabled)

Parent / Player (mobile-primary):

- Home (next event + RSVP prompt)
- Schedule (my child's / my games)
- RSVP
- Standings & team page
- Notifications
- Registration & payment

Public (no login):

- Public schedule
 - Public standings
 - Public bracket
 - Team public page
-

5. Data model

Multi-tenant: nearly every table carries an `organization_id`. IDs are UUIDs. Timestamps (`created_at` , `updated_at`) on all tables; soft-delete (`deleted_at`) where useful.

5.1 Core entities

Organization — the tenant/account. `name` , `slug` , `logo_url` , `brand_color` , `timezone` , `subscription_tier` , `stripe_customer_id` , `org_type` (`league_operator` | `club` | `tournament_host` — an org may wear more than one hat). This separates the *operator* of a competition from the *clubs* that field teams.

User — a person (global, can belong to many orgs). `email` (unique, the identity anchor), `email_verified_at` , `name` , `phone` , `avatar_url` , `password_hash?` (null for users who only sign in via Google), `status` (`active` | `invited` | `disabled`), `last_login_at` . A user may authenticate by password, magic link, or one or more linked providers — see `AuthIdentity` .

AuthIdentity — a linked sign-in method for a User. `user_id` , `provider` (`google` | `apple` | `password` | `magic_link`), `provider_account_id` (e.g. Google `sub`), `email_at_provider` , `created_at` . Unique on (`provider` , `provider_account_id`) . This lets one account hold several login methods (Google **and** password) and makes "Sign in with Google" first-class rather than a flag on User.

Membership — join of User ↔ Organization with `role` (`owner` | `admin` | `coach` | `scorekeeper` | `parent` | `player`) and optional `scope` (e.g. team or club IDs a coach manages). A user can have multiple memberships per org.

Sport — `name` , `stat_schema` (JSON describing trackable stats), `default_scoring_rules` , `tiebreaker_rules` . Seeded library (soccer, basketball, baseball, etc.) + custom.

Club — an entity that fields teams across age groups and divisions. `organization_id?` (a club may be its own tenant org, or a lightweight record in a league operator's registry), `name` , `logo_url` , `colors` , `home_venue_id?` , `contact_user_id` . A club can enter teams into many competitions run by *other* organizations.

Team — belongs to a **Club**, and is identity-stable across competitions/seasons. `club_id` , `name` , `age_group` , `gender` , `logo_url` , `color` , `coach_user_id` , `external_ref?` . A team reaches competitive play by entering divisions (see `DivisionEntry`), not by being owned by one.

Season — a time-scoped operating period owned by the Organization (e.g. "Fall 2026"), and the **parent of competitions**. Holds shared defaults that every competition inside it inherits: `registration_opens_at` / `registration_closes_at` , `age_cutoff_date` (age-eligibility rule), `timezone` , `blackout_dates` , default ruleset. `organization_id` , `name` , `start_date` , `end_date` , `status` . One season contains many competitions running concurrently (U8/U10/U12 leagues + a tournament).

Competition — a competitive container run within a Season. `organization_id` , `season_id?` (nullable — a standalone tournament host can skip seasons), `series_id?` (see below), `sport_id` , `name` , **`type` (`league` | `tournament`)**, `start_date` , `end_date` , `status` (`draft` | `registration` | `active` | `completed`), `format_config`

(JSON — round-robin rounds & points rules for leagues; bracket format, pool sizes & seeding method for tournaments). See §5.6 for how the two types differ.

CompetitionSeries (*optional*) — links recurring editions of the same competition across seasons (e.g. the Premier League each fall) for cross-season history and records. `organization_id`, `name`; referenced by `Competition.series_id`. This is the "recurring brand" axis, kept separate from Season (the "what's happening now" axis) so neither is overloaded.

Division — belongs to a **Competition**. `competition_id`, `name`, `age_group`, `skill_level`, `gender`, `ruleset` (points-per-win, tiebreakers). For tournaments, divisions contain stages (see §5.3).

DivisionEntry — join of **Team** ↔ **Division** (a team registered into a division for a competition). `team_id`, `division_id`, `seed?` (tournaments), `pool_id?`, `status` (`registered|confirmed|withdrawn`). This is the key relationship: one club → many teams → each team enters one or more divisions across leagues *and* tournaments.

Player — a youth athlete profile. `first_name`, `last_name`, `birthdate`, `gender`, `photo_url?` (uploaded headshot; guardian-consent flag + visibility controls per §9), `user_id?` (linked account if old enough). Linked to guardians.

GuardianLink — Player ↔ User (guardian) with `relationship`.

RosterEntry — Player ↔ Team for a competition/season. `jersey_number`, `position`, `status` (`active|injured|inactive`).

5.2 Scheduling

Venue — belongs to Org. `name`, `address`, `lat`, `lng`, `timezone`.

Field — belongs to Venue. `name`, `surface`, `capacity_slots`.

Event — base schedule item. `type` (`game|practice|tournament_game|other`), `division_id?`, `field_id?`, `starts_at`, `ends_at`, `status` (`scheduled|in_progress|final|canceled|postponed`), `notes`.

Game — extends Event for competitive play. `home_team_id`, `away_team_id`, `home_score`, `away_score`, `period_scores` (JSON), `scorekeeper_user_id?`, `tournament_match_id?`.

GameEvent — discrete scoring/stat events for a game. `game_id`, `player_id?`, `team_id`, `type` (`goal|assist|foul|sub|...`), `period`, `clock`, `value`.

Availability (RSVP) — `event_id`, `player_id` (or `user_id`), `status` (`yes|no|maybe|no_response`), `responded_at`, `note`.

5.3 Compete

A `Competition` with `type = tournament` adds the structures below. (A `type = league` competition skips these and uses round-robin `Games` + cumulative `Standings`.)

Stage — belongs to a Division (within a tournament competition). `division_id`, `type` (`pool|bracket`), `format` (`single_elim|double_elim|round_robin`), `order`, `seeding_method` (`manual|by_record|by_ranking`).

Pool — belongs to a pool-stage. `stage_id`, `name`; teams assigned via `DivisionEntry.pool_id`. Pool standings feed seeding into the next bracket stage.

Match — a slot in a bracket/pool. `stage_id`, `round`, `position`, `home_source` / `away_source` (refs winner/loser of another match, a seed, or a pool finish), `game_id?`. Auto-advances as linked games are finalized.

Standing — computed cache per Team per Division/Pool. `wins`, `losses`, `ties`, `points`, `points_for`, `points_against`, `rank`, `tiebreaker_data` (JSON). Recomputed on game finalization.

PlayerStat — aggregated per Player per Season (and per Game). Shape driven by `Sport.stat_schema`.

5.4 Engagement & ops

Registration — `season_id`, `player_id`, `team_id?`, `status` (`pending|confirmed|waitlist`), `form_data` (JSON), `waiver_signed_at`.

Payment — `organization_id`, `user_id`, `registration_id?`, `amount`, `currency`, `status`, `stripe_payment_intent_id`.

Announcement / Message — `scope` (`org|division|team`), `author_user_id`, `title`, `body`, `audience_filter`.

Notification — per-user delivery record. `user_id`, `channel` (`email|push|sms`), `type`, `payload`, `read_at`, `sent_at`.

ShareLink — public token for a resource. `resource_type` (`schedule|standings|bracket|team`), `resource_id`, `token`, `expires_at?`.

AuditLog — `organization_id`, `actor_user_id`, `action`, `target`, `metadata`, `created_at`.

5.5 Relationship map (text ER)

```
Organization 1—* Membership *—1 User
Organization 1—* Club 1—* Team 1—* RosterEntry *—1 Player
Player *—* User (via GuardianLink)
Player 1—1 photo_url (guardian-consent gated)

Organization 1—* Season 1—* Competition (type = league | tournament) 1—1 Sport
CompetitionSeries 1—* Competition (optional: recurring editions across seasons)
Competition 1—* Division 1—* DivisionEntry *—1 Team (a team enters a division)
  └ [league] Division 1—* Game → Standings
  └ [tournament] Division 1—* Stage 1—* (Pool | Match) -?- Game → advancement

Division 1—* Event ⊂ Game (home_team, away_team → Team via DivisionEntry)
Venue 1—* Field 1—* Event
Event 1—* Availability *—1 Player
Game 1—* GameEvent
Team 1—1 Standing (per Division / Pool)
Season 1—* Registration 1—? Payment (season-level signup → division placement)
```

A Club's teams fan out across many divisions and competitions; a single Team can simultaneously hold a `DivisionEntry` in a season-long **league** and another in a weekend **tournament**.

5.6 League vs Tournament — the delineation

Both are `Competition` rows sharing the same teams, games, scoring, RSVP, and public pages — but they behave differently:

Duration	Weeks–months, often recurring (Seasons)	Time-boxed event (a day / weekend)
Structure	Division → Games	Division → Stage(s) → Pools/Matches
Schedule	Round-robin / balanced, generated across rounds	Pool play and/or single/double-elim bracket
Progression	Cumulative standings over the season	Elimination & auto-advancement
Seeding	Not required (optional for playoffs)	Required — manual, by record, or by ranking
Outcome	Champion by points + tiebreakers	Bracket winner
Primary view	Standings table	Bracket + pool sheets

This is why `type` lives on `Competition` (not as a flag on `Division`): scheduling, progression, and the public surface all branch on it.

6. API design

Style: REST + JSON over HTTPS, versioned at `/api/v1`. Resource-oriented, plural nouns. Realtime via WebSocket channels for live game/standings updates. Stripe & email/push via webhooks.

6.1 Conventions

- **Auth:** Bearer JWT (short-lived access + refresh). Org context via `X-Org-Id` header or `/orgs/{orgId}/...` path scoping.
- **Tenancy:** every org-scoped route nested under `/orgs/{orgId}`. Server enforces `membership(user, org)` + role on every request.
- **Pagination:** cursor-based — `?limit=&cursor=`. Responses: `{ data: [...], next_cursor }`.
- **Filtering/sorting:** `?filter[status]=active&sort=-starts_at`.
- **Errors:** RFC 7807 problem+json — `{ type, title, status, detail, errors[] }`.
- **Idempotency:** `Idempotency-Key` header on POST for payments & score submits.

6.2 Resource map (selected)

```
# Auth & identity
POST /api/v1/auth/register # email + password sign-up
POST /api/v1/auth/login # email + password
POST /api/v1/auth/magic-link # request a one-time email link
POST /api/v1/auth/verify-email # confirm email via token
POST /api/v1/auth/password/forgot # request reset
POST /api/v1/auth/password/reset # set new password via token
POST /api/v1/auth/refresh # rotate access token
POST /api/v1/auth/logout # revoke refresh token

# Google OAuth (Authorization Code + PKCE)
GET /api/v1/auth/google/start # redirect to Google consent
GET /api/v1/auth/google/callback # exchange code → session (sign-up or login)

# Current user & account/profile management
GET /api/v1/me # profile + linked identities
PATCH /api/v1/me # update name, phone, avatar
GET /api/v1/me/memberships
GET /api/v1/me/identities # list linked sign-in methods
POST /api/v1/me/identities/google # link Google to existing account
DELETE /api/v1/me/identities/{provider} # unlink (block if it's the only method)
DELETE /api/v1/me # delete account (GDPR/CCPA)

# Organizations
POST /api/v1/orgs
GET /api/v1/orgs/{orgId}
PATCH /api/v1/orgs/{orgId}
GET /api/v1/orgs/{orgId}/members
POST /api/v1/orgs/{orgId}/members # invite
PATCH /api/v1/orgs/{orgId}/members/{userId} # change role

# Clubs & teams (a club owns teams across age groups)
GET /api/v1/orgs/{orgId}/clubs
POST /api/v1/orgs/{orgId}/clubs
GET /api/v1/orgs/{orgId}/clubs/{clubId}/teams
POST /api/v1/orgs/{orgId}/clubs/{clubId}/teams
GET /api/v1/orgs/{orgId}/teams/{teamId}
GET /api/v1/orgs/{orgId}/teams/{teamId}/roster
POST /api/v1/orgs/{orgId}/teams/{teamId}/roster # add player

# Competitions (league | tournament) → divisions → entries
GET /api/v1/orgs/{orgId}/competitions?filter[type]=league
POST /api/v1/orgs/{orgId}/competitions # body: { type: "tournament", ... }
GET /api/v1/orgs/{orgId}/competitions/{compId}/divisions
POST /api/v1/orgs/{orgId}/competitions/{compId}/divisions
POST /api/v1/orgs/{orgId}/divisions/{divisionId}/entries # enter a team (DivisionEntry, with roster)
GET /api/v1/orgs/{orgId}/divisions/{divisionId}/entries

# Players & guardians
POST /api/v1/orgs/{orgId}/players
```

```

POST /api/v1/orgs/{orgId}/players/{playerId}/photo # upload headshot (multipart)
POST /api/v1/orgs/{orgId}/players/{playerId}/guardians

# Venues & schedule
GET /api/v1/orgs/{orgId}/venues
POST /api/v1/orgs/{orgId}/venues/{venueId}/fields
GET /api/v1/orgs/{orgId}/events?filter[type]=game&from=&to=
POST /api/v1/orgs/{orgId}/events
PATCH /api/v1/orgs/{orgId}/events/{eventId} # reschedule/cancel
POST /api/v1/orgs/{orgId}/divisions/{divisionId}/schedule:generate # auto-scheduler

# RSVP
GET /api/v1/orgs/{orgId}/events/{eventId}/availability
PUT /api/v1/orgs/{orgId}/events/{eventId}/availability/{playerId}

# Game day / scoring
POST /api/v1/orgs/{orgId}/games/{gameId}/score # update score (idempotent)
POST /api/v1/orgs/{orgId}/games/{gameId}/events # log goal/foul/etc.
POST /api/v1/orgs/{orgId}/games/{gameId}:finalize # locks result, recomputes standings

# Compete (tournament-type competitions)
POST /api/v1/orgs/{orgId}/divisions/{divisionId}/stages # add pool/bracket stage
POST /api/v1/orgs/{orgId}/divisions/{divisionId}:seed # seed entries
GET /api/v1/orgs/{orgId}/divisions/{divisionId}/bracket
POST /api/v1/orgs/{orgId}/matches/{matchId}:advance # propagate winner

# Insights
GET /api/v1/orgs/{orgId}/divisions/{divisionId}/standings
GET /api/v1/orgs/{orgId}/teams/{teamId}/stats
GET /api/v1/orgs/{orgId}/players/{playerId}/stats

# Registration & payments
POST /api/v1/orgs/{orgId}/registrations
POST /api/v1/orgs/{orgId}/payments/intent
POST /api/v1/webhooks/stripe

# Public (no auth, token-scoped)
GET /api/v1/public/{shareToken}/schedule
GET /api/v1/public/{shareToken}/standings
GET /api/v1/public/{shareToken}/bracket

```

6.3 Realtime channels (WebSocket)

- `org:{orgId}:game:{gameId}` — score & event stream
- `org:{orgId}:division:{divisionId}:standings` — standings recompute
- `org:{orgId}:tournament:{id}` — bracket advancement
- `user:{userId}:notifications` — push/in-app

6.4 Sample payloads

POST /games/{gameId}/score

```
{ "home_score": 2, "away_score": 1, "period": 2, "clock": "12:30" }
```

GET /divisions/{id}/standings →

```
{
  "division_id": "...",
  "updated_at": "2026-06-14T18:02:00Z",
  "standings": [
    { "rank": 1, "team": "Comets", "w": 6, "l": 1, "t": 1, "pts": 19, "pf": 24, "pa": 9 }
  ]
}
```

6.5 Authentication & user management

Sign-in methods (all map to one `User` via `AuthIdentity`):

- **Email + password** — register, verify email, login, forgot/reset password.
- **Magic link** — passwordless one-time email link (good for parents/players who rarely log in).
- **Sign in with Google** — OAuth 2.0 Authorization Code + PKCE; primary social option.

Google sign-up / login flow:

1. User clicks **Continue with Google** → GET /auth/google/start redirects to Google consent.
2. Google redirects back to GET /auth/google/callback with a code; server exchanges it for tokens and reads the verified profile (`sub` , `email` , `name` , `picture`).
3. **Match logic**: if an `AuthIdentity` exists for that Google `sub` → log in. Else if a `User` exists with that **verified** email → link Google to it (then log in). Else → create a new `User` + `AuthIdentity` (sign-up). Google emails are treated as pre-verified.
4. Issue our own short-lived access JWT + rotating refresh token (httpOnly cookie); we never persist Google tokens beyond the exchange.

Account linking & safety:

- A user can link **both** Google and a password to the same account (POST /me/identities/google).
- Unlinking is blocked if it would leave the account with **zero** sign-in methods.
- Email is the identity anchor and must be verified before two methods auto-merge (prevents account-takeover via unverified email).

Sessions & security: short-lived access token + refresh-token rotation with reuse detection; POST /auth/logout revokes the refresh token. Rate-limit login / magic-link / reset endpoints. Optional 2FA (TOTP) is a later add.

Profile/account management: users edit name, phone, and avatar via `PATCH /me` ; view and manage linked sign-in methods; and self-serve account deletion (`DELETE /me`) for GDPR/CCPA. Org **membership/role** management stays separate (admins manage it under `/orgs/{orgId}/members`) — authentication (who you are) vs. authorization (what you can do per org).

Minors: youth players are `Player` profiles managed by guardians, not self-service `User` accounts by default; a player only gets a login (and thus an auth method) when old enough and guardian-approved.

7. Architecture & tech stack

Layer	Choice	Why
Web client	Next.js (React) + TypeScript , Tailwind CSS, Sora font	SSR for fast public pages + SEO; PWA for mobile; shared design system
State/data	TanStack Query + WebSocket client	Cache + realtime
API	NestJS (Node + TS) REST, or Next route handlers for a monolith start	Typed, modular, guards for RBAC
Database	PostgreSQL + Prisma ORM	Relational fit for leagues/brackets; JSON columns for flexible stat schemas
Cache / realtime	Redis (pub/sub + cache)	Standings cache, WS fan-out, rate limiting
Auth	JWT (access) + rotating refresh, email/password, magic link, Google OAuth (Auth Code + PKCE) , Apple later; <code>AuthIdentity</code> table for linked providers	Familiar, supports SSO + account linking
Payments	Stripe (bones in v1, live in fast-follow)	Registration fees + subscriptions
Notifications	Resend (email) + Web Push (VAPID); SMS (Twilio) later	Multi-channel, no SMS at launch
Files	S3-compatible object storage	Logos, avatars, exports
Hosting	Vercel (web) + managed Postgres (Neon/Supabase) + Redis (Upstash)	Low ops to start
Observability	Sentry + structured logs + uptime checks	Game-day reliability matters

Architecture posture: start as a **modular monolith** (one deployable API, clear module boundaries: identity, scheduling, scoring, compete, billing, notifications). Extract services only if/when scale demands. Multi-tenancy

enforced at the query layer (every query filtered by `organization_id` ; consider Postgres row-level security as a backstop).

Design system: build a small component library themed from the brand kit — gradient (`#7B2DFF` → `#FF2D72` → `#FF8A00` → `#FFC700`) reserved for the mark, primary CTAs, and accents; Sora for type; neutrals carry layout. The brand sheet in `Brand-Kit/` is the source of truth. **Mobile-first and fully responsive is a hard requirement** — every screen must work cleanly on phone, tablet, and desktop (admin tools are desktop-primary but must remain usable on tablet). Per-org branding is **logo + color only**, layered on the PlayPulse base.

7.1 Environments & deployment workflow

We build on a `dev` branch/environment and ship to production by merging to `main` .

Environment	Branch	Hosting	Database	Purpose
Development	<code>dev</code>	Vercel preview / dev deployment	Neon <code>dev</code> branch (seeded test data)	Active build-out and integration; safe to break
Production	<code>main</code>	Vercel production	Neon <code>main</code> (real data, backups on)	Live app
<i>Staging (optional, later)</i>	<code>staging</code>	Vercel	Neon <code>staging</code>	Pre-release smoke test once there are real users

Branching & merge flow:

- 1. Branch feature work off `dev` (`feat/...`), open a PR into `dev` .
- 2. CI runs on every PR — lint, typecheck, unit/integration tests, build. Green required to merge.
- 3. Merge to `dev` → auto-deploy to the Development environment for review.
- 4. When a slice is stable, open a PR `dev` → `main` ; on merge, auto-deploy to Production.
- 5. `main` is protected: no direct pushes, PR + passing CI (and review once there's a team) required.

Config & secrets: per-environment env vars (`DATABASE_URL` , `REDIS_URL` , Google OAuth client id/secret + redirect URI, Resend key, Stripe keys later) stored in Vercel's environment settings — never committed. Each environment has its **own** Google OAuth redirect URLs and its own DB so dev work never touches production data.

Migrations: Prisma migrations run automatically on deploy (dev first, then main). Destructive migrations are reviewed before they reach `main` .

Releases & rollback: `main` deploys are the release boundary; tag notable releases. Vercel keeps prior deployments for instant rollback, and Neon point-in-time restore covers the database.

8. Phased roadmap

Phase 0 — Foundations (≈2 wks) Repo, CI/CD, design system from brand kit, **auth (email/password, magic link, Google OAuth, email verification, password reset)** + **User / AuthIdentity** model, org + membership + RBAC, account/profile screens, base layouts.

Phase 1 — Manage (≈3–4 wks) Seasons/divisions/teams, players & guardians, rosters, registration (no payment yet), member invites.

Phase 2 — Schedule (≈3–4 wks) Venues/fields, manual schedule editor, auto-scheduler, conflict detection, RSVP, calendar feeds, notifications (email + push).

Phase 3 — Game day & Insights (≈3 wks) Live score entry, game events, finalize → standings recompute, realtime channels, public schedule/standings pages.

Phase 4 — Compete (≈3 wks) Tournament engine (formats, seeding, auto-advance), bracket UI, public bracket pages.

Phase 5 — Monetize & polish (≈2–3 wks) (*fast-follow*) Activate Stripe registration payments + subscriptions (on the v1 bones), reporting/exports, player stats.

Phase 6 — v2 candidates Additional sports, SMS, native apps, league discovery, officials assigning/payments, advanced analytics.

Logo + color org branding and full phone/tablet/desktop responsiveness are built into Phases 0–4, not deferred.

9. Non-functional requirements

- **Security:** RBAC on every endpoint, tenant isolation, signed share-link tokens, COPPA-aware handling of minors' data (guardian consent, minimal PII, no public child contact info), encrypted at rest/in transit, PCI handled via Stripe.
- **Privacy:** youth data minimization; public pages never expose birthdates or contact info; configurable visibility.
- **Performance:** public pages SSR/cached < 1s; standings recompute < 2s after finalize; schedule generation async with progress.
- **Reliability:** game-day is peak load — score entry must work on flaky stadium wifi (optimistic UI, retry, offline queue).
- **Accessibility:** WCAG 2.1 AA, keyboard nav, sufficient contrast (mind the gradient on text — never set body text in gradient).

- **i18n-ready:** copy externalized; timezone-correct everywhere (events store UTC + venue tz).
-

10. Decisions (locked 2026-06-14)

1. **Sport at launch — soccer only.** Build the `Sport.stat_schema` abstraction so other sports slot in later, but ship and tune for soccer alone.
 2. **Payments — build the bones, defer the flow.** Model `Registration` + `Payment` and the season-level signup → division-placement path now; wire live Stripe checkout as a fast-follow.
 3. **Single-org, no marketplace.** Sold to individual orgs. No public league discovery in v1 — later.
 4. **Notifications — email (Resend) + web push.** No SMS at launch.
 5. **PWA, no native app.** All screens fully responsive and mobile-friendly across phones and tablets (and desktop). App-store presence is a later consideration.
 6. **Branding — logo + color only.** Per-org logo and brand color layered on the PlayPulse base; no full per-org theming.
-

This plan auto-renders to `PLAN.pdf` on every edit (`PostToolUse` hook → `scripts/plan-to-pdf.sh`).